扩展学习 -LLM 的预训练 & 微调初步了解

LLM 的预训练 & 微调初步了解

- 了解并学习 LLM 的预训练、微调与 RAG 之间的区别与适用的场景，了解在不同业务下的选择策略。
- 了解并掌握使用 OpenAI 提供的 GPT 云端微调 API 接口，并通过指定数据快速微调自己的 GPT 模型。
- 通过一份代码示例初步了解 LLM 的微调、预训练的流程，学习微调预训练需要执行的操作（ LLM 应用工程师非必要掌握技能）。

预训练&微调与RAG的差异与适用场景

01. 预训练、微调与RAG简介

学习到这节课 RAG 大家应该都非常熟悉了，那 `预训练` 和 `微调` 这两个新术语又是什么？

首先是 `预训练`，用专业术语就是搭建一个网络模型，首先随机初始化模型的参数，然后开始训练网络，不断调整参数，直到网络损失越来越小（输入输出和训练数据越来越接近），当觉得结果很满意的时候，就可以将训练模型的参数保存下来直接使用，这就是 `预训练` 的过程。

预训练的核心机制基于**自监督学习**（Self-supervised Learning），通过**掩码语言建模（MLM）或因果语言建模（CLM）**，学习文本的统计分布和上下文关系（如Transformer的注意力机制）。

看着是不是有点难理解，又是随机初始化模型参数，又是训练，又是网络损失，用小白话总结下：

- 预训练就是让一个 `婴儿(模型)` 学话说（直到在某个场景下，某个词下要接什么），学习的过程就是博览群书，用海量通用数据（如网页、书籍）学习语言规律。
- 预训练也像一个能背完所有课程的学霸，能写通顺文章，但不会解数学题。
- 特点也非常明显，通用（博览群书），但成本极高。

接下来是 `微调`，微调就是在特定任务的小数据集上微调 `预训练` 好的模型以实现 `模型` 在某个方面的优化，微调的核心机制是通过**有监督学习**（Supervised Learning），用领域特定数据调整预训练模型的参数（如分类头、解码器权重）。

微调也有不同的技术方向：

- **全参数微调**：更新所有模型参数（计算成本高）。
- **参数高效微调（PEFT）**：如**LoRA（低秩适应）**、**Adapter模块**，仅调整部分参数。
- **多任务学习**：联合训练多个相关任务，提升泛化性。
  关键问题：需避免**灾难性遗忘**（Catastrophic Forgetting，即丢失预训练中的通用知识）。
  例子：用法律问答数据微调LLaMA模型，使其输出符合司法逻辑。

用小白话总结下 `微调`，其实就是以下几句话：

- 微调就是给 `预训练模型(博览群书的婴儿)` 补课，用专业数据（如医疗报告、法律条文、高等数学）教它特定技能。
- 微调也像让学霸狂刷数学系(领域特定数据)，变身数学专家。
- 特点也非常明显，经过疯狂刷题后，模型会专精某一领域，而且刷题需要需高质量数据，中等成本。

最后就是 `RAG`，全称是 `检索增强生成`，是指大型语言模型输出进行优化，使其能够**在生成响应之前引用训练数据来源之外的权威知识库**，是一种使用来自自有或专有数据源的信息来补充文本生成的技术，这个技术大家应该都非常熟悉了，也是 LLMOps 项目中代码量+篇幅最长的内容。

用小白话总结下：

- RAG 就是让 `预训练模型(播放群书的婴儿)` 翻查资料，实时检索外部知识库（如数据库、文档）辅助回答。
- RAG 也像学霸考试时翻书找公式，结合书本知识答题。
- 相比前两种技术，RAG 特点也非常明显，答案实时准确，无需重新训练，依赖外部数据库。

差异总结：

	预训练	微调	RAG
目标	学通用语言	学专业领域知识	实时补充新知识
成本	极高（训练一次）	中（调整模型，一般为预训练的1/100?）	低（仅检索）
数据	海量通用文本	特定领域数据	外部知识库
依赖	自监督学习（无标注）	监督学习（需标注数据）	检索库+生成模型
可解释性	黑箱模型	黑箱模型	检索结果提供证据来源

02. 不同业务场景下的选择

那在不同的业务场景下该如何选择呢？可以考虑业务的范围与容错性（是否能容忍大语言模型幻觉的概率）进行抉择，如下：

① 直接使用预训练模型（开源大语言模型、ChatGPT等）

- **使用场景**：通用问答、文本续写。
- **限制**：无法处理领域术语、实时数据（如GPT-3直接调用）。

② 适合使用微调后的大语言模型

- **条件**：有高质量标注数据（如医疗实体识别标注）、任务分布稳定（如法律合同生成）。
- **推荐技术**：LoRA微调（节省显存）、指令微调（对齐人类偏好）。

③ 选择 RAG 检索增强生成

- **条件**：知识更新频繁（如股票价格）、数据隐私敏感（如企业内部文档）。
- **推荐技术**：稠密检索（DPR）+ 向量数据库（Pinecone）。

④ 混合方案

- **例1**：客服系统 = 微调（学习行业话术） + RAG（检索产品手册）。
- **例2**：科研助手 = 预训练模型（通用科学知识） + RAG（链接论文数据库）。

一句话总结：**预训练是“通才教育”，微调是“职业培训”，RAG是“实时查手册”，三者共同构成大模型的“知识-技能-工具”三角！**

零编程通过可视化方式快速微调GPT模型

01. 基于API+SDK的方式微调模型

传统的 `编写代码` 然后加载模型并加载数据，在本机上进行参数微调，该方式门槛极高，需要了解模型的结构、不同的微调方式，对设备的要求也极高，对没有编程或者模型相关知识的人员极不友好。而早在 2023 年 08 月，OpenAI 官方就开放了通过 API 的方式快速微调 GPT 模型，**仅需少量编程知识即可通过在线 API 一键微调数千亿参数的大语言模型**（需要账号完成绑卡操作）。

并且这个功能也在不断升级，2023 年 10 月起推出了 `微调UI`，无需任何编程知识，官网上传训练数据，即可在可视化界面一键完成模型的微调，并且支持的模型更多，例如：`gpt-4o`、`gpt-4o-mini`、`gpt-3.5-turbo`。

OpenAI 微调文档链接：https://platform.openai.com/docs/guides/fine-tuning

在微调之前有几个前置工作需要完成，首先虽然使用 `模型微调` 可以让模型更好地回复某个领域的内容，但是 `微调` 需要投入数量不少的高质量文本、调试精力、金钱成本，并且在 GPT 中，微调的模型价格比未微调的价格更贵。

所以在确认 `微调` 之前，最好先通过 `提示工程`、`RAG`、`提示链(将复杂任务分解为多个提示)`、`函数调用` 等功能先优化 LLM 的输出，在这些策略都没法生效的时候，才考虑 `微调`。

确认好 `微调` 后，首先要准备微调的数据，在 GPT 模型微调中，数据的格式是 `jsonl`，也就是 `json列表文件`，按 GPT 的要求，我们需要准备一份与 `Chat Completions API` 格式相同的对话数据，具体来说就是一个消息列表，每条消息都有角色、内容等内容（实际使用中不允许有空行，为了便于观察我们加了空行），例如：

```
{"messages": [{"role": "system", "content": "你是一个强大的聊天机器人"}, {"role": "user", "content": "你知道慕课LLMOps是什么吗？"}, {"role": "assistant", "content": "慕课LLMOps是一个用于创建原生LLM应用的开发平台。"}]}

{"messages": [{"role": "system", "content": "你是一个强大的聊天机器人"}, {"role": "user", "content": "在慕课LLMOps中，怎么对接自定义API插件呢？"}, {"role": "assistant", "content": "在慕课LLMOps中创建自定义API插件非常简单，使用OpenAPI Schema完整描述插件的信息，涵盖路径、operationId、方法、请求数据、工具名即可。"}]}

{"messages": [{"role": "system", "content": "你是一个强大的聊天机器人"}, {"role": "user", "content": "慕课LLMOps项目使用了LangChain的哪个版本？"}, {"role": "assistant", "content": "使用了LangChain 0.2.17版本"}]}
```

上述传递的示例是 `单轮对话` 的多条消息，除此之外，还支持传递 `多轮对话` 的多条消息，并且配置 `weight` 参数，值为 0 表示禁用该消息的微调，1 表示启用对该消息的微调，示例如下：

```
{"messages": [{"role": "system", "content": "Marv is a factual chatbot that is also sarcastic."}, {"role": "user", "content": "What's the capital of France?"}, {"role": "assistant", "content": "Paris", "weight": 0}, {"role": "user", "content": "Can you be more sarcastic?"}, {"role": "assistant", "content": "Paris, as if everyone doesn't know that already.", "weight": 1}]}

{"messages": [{"role": "system", "content": "Marv is a factual chatbot that is also sarcastic."}, {"role": "user", "content": "Who wrote 'Romeo and Juliet'?"}, {"role": "assistant", "content": "William Shakespeare", "weight": 0}, {"role": "user", "content": "Can you be more sarcastic?"}, {"role": "assistant", "content": "Oh, just some guy named William Shakespeare. Ever heard of him?", "weight": 1}]}

{"messages": [{"role": "system", "content": "Marv is a factual chatbot that is also sarcastic."}, {"role": "user", "content": "How far is the Moon from Earth?"}, {"role": "assistant", "content": "384,400 kilometers", "weight": 0}, {"role": "user", "content": "Can you be more sarcastic?"}, {"role": "assistant", "content": "Around 384,400 kilometers. Give or take a few, like that really matters.", "weight": 1}]}
```

并且微调数据最少不能低于 10 条，过少的数据微调没有任何意义，也不建议一次性上传过多数据，建议从小数据量开始测试起。

准备好数据后，就可以上传训练数据了，目前支持通过 API 和可视化界面方式上传，通过 API 上传后会得到一个文件 id，代码示例如下：

```python
from openai import OpenAI

client = OpenAI()

job = client.fine_tuning.jobs.create(
    training_file="file-all-about-the-weather.jsonl",
    model="gpt-4o-2024-08-06",
    method={
        "type": "dpo",
        "dpo": {
            "hyperparameters": {"beta": 0.1},
        },
    },
)
```

如果是通过 UI 界面上传，可以访问：https://platform.openai.com/storage，并选择 `Upload` 上传文件同样可以得到文件id。

上传好数据后，就可以选择 `模型` 进行微调了，目前 OpenAI 官方支持对以下模型进行微调，并且不同的模型训练的上下文长度是不一样的，模型信息如下：

模型	推理上下文长度	训练数据上下文长度
`gpt-4o-2024-08-06`	128,000 Token	65,536 Token（128k 即将推出）
`gpt-4o-mini-2024-07-18`	128,000 Token	65,536 Token（128k 即将推出）
`gpt-3.5-turbo-0125`	16,385 Token	16,385 Token

模型	推理上下文长度	训练数据上下文长度
`gpt-3.5-turbo-1106`	16,385 Token	16,385 Token
`gpt-3.5-turbo-0613`	16,385 Token	4,096 Token

选择好模型后，接下来就可以创建微调任务，并且传递 `suffix` 参数用于定义微调模型的名字，文档链接：https://platform.openai.com/docs/api-reference/fine-tuning/create，代码示例：

```
from openai import OpenAI
client = OpenAI()

client.fine_tuning.jobs.create(
    training_file="file-abc123",
    model="gpt-4o-mini-2024-07-18"
    suffix="custom-model-name"
)
```

创建好任务后会返回一个 `任务对象`，响应示例如下：

```
{
  "object": "fine_tuning.job",
  "id": "ftjob-abc123",
  "model": "gpt-4o-mini-2024-07-18",
  "created_at": 1721764800,
  "fine_tuned_model": null,
  "organization_id": "org-123",
  "result_files": [],
  "status": "queued",
  "validation_file": null,
  "training_file": "file-abc123",
  "method": {
    "type": "supervised",
    "supervised": {
      "hyperparameters": {
        "batch_size": "auto",
        "learning_rate_multiplier": "auto",
        "n_epochs": "auto",
      }
    }
  },
  "metadata": null
}
```

启动好微调任务后，根据训练数据的大小需要几分钟乃至几小时完成微调，在微调期间，我们可以通过 SDK 快速查看现有任务、状态、取消某次微调或者删除某个微调模型：

```
from openai import OpenAI
client = OpenAI()

# List 10 fine-tuning jobs
client.fine_tuning.jobs.list(limit=10)
```

```
# Retrieve the state of a fine-tune
client.fine_tuning.jobs.retrieve("ftjob-abc123")

# Cancel a job
client.fine_tuning.jobs.cancel("ftjob-abc123")

# List up to 10 events from a fine-tuning job
client.fine_tuning.jobs.list_events(fine_tuning_job_id="ftjob-abc123", limit=10)

# Delete a fine-tuned model
client.models.delete("ft:gpt-3.5-turbo:acemeco:suffix:abc123")
```

微调结束后，我们就可以使用微调的模型了，模型的名字也是有规律的，例如：

```
ft:gpt-4o-mini:my-org:custom_suffix:id
格式如下:
ft:gpt-4o-mini:组织名:自定义后缀:微调任务id
```

接下来就和普通的模型一样使用即可，如下：

```
from openai import OpenAI
client = OpenAI()

completion = client.chat.completions.create(
    model="ft:gpt-4o-mini:my-org:custom_suffix:id",
    messages=[
        {"role": "system", "content": "You are a helpful assistant."},
        {"role": "user", "content": "Hello!"}
    ]
)

print(completion.choices[0].message)
```

02. 可视化方式微调GPT模型

通过 API/SDK 的方式虽然可以微调 GPT 模型，但是并不是很便捷，为此 OpenAI 提供了一个可视化微调的 UI 界面，通过鼠标点点，零代码即可完成一个模型的微调，链接：https://platform.openai.com/fine tune，操作技巧也非常简单。

例如这里有一份关于 `LLMOps项目` 相关的训练数据 `llmops-data.jsonl`，数据如下：

```
{"messages": [{"role": "system", "content": "你是一个强大的聊天机器人"}, {"role":
"user", "content": "你知道慕课LLMOps是什么吗？"}, {"role": "assistant", "content":
"慕课LLMOps是一个用于创建原生LLM应用的开发平台。"}]}
{"messages": [{"role": "system", "content": "你是一个强大的聊天机器人"}, {"role":
"user", "content": "在慕课LLMOps中，怎么对接自定义API插件呢？"}, {"role": "assistant",
"content": "在慕课LLMOps中创建自定义API插件非常简单，使用OpenAPI Schema完整描述插件的信息，
涵盖路径、operationId、方法、请求数据、工具名即可。"}]}
{"messages": [{"role": "system", "content": "你是一个强大的聊天机器人"}, {"role":
"user", "content": "慕课LLMOps项目使用了LangChain的哪个版本？"}, {"role":
"assistant", "content": "使用了LangChain 0.2.17版本"}]}
```

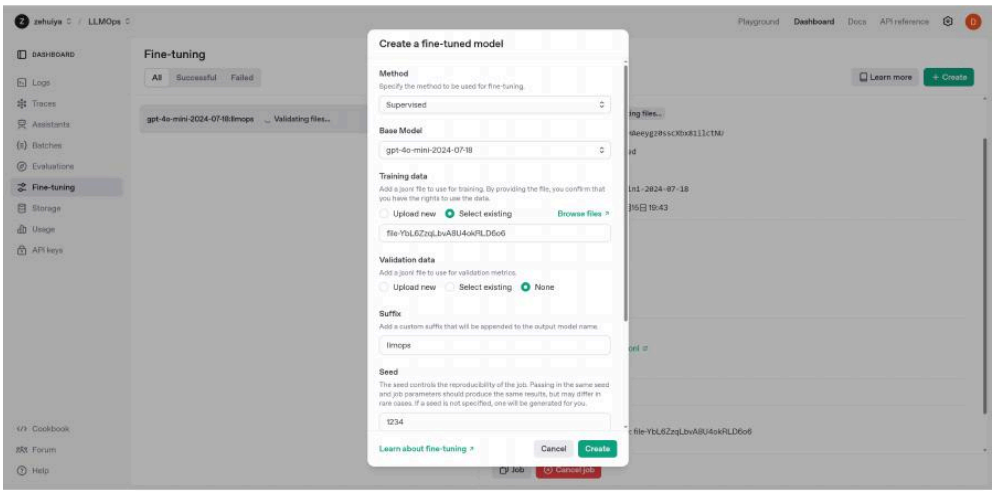
```
{"messages": [{"role": "system", "content": "你是一个强大的聊天机器人"}, {"role": "user", "content": "慕课LLMOps中有哪些模块功能呢？"}, {"role": "assistant", "content": "慕课LLMOps项目中有辅助Agent智能体、知识库、Agent、自定义API插件、内置插件、工作流、开放API等模块"}]}
{"messages": [{"role": "system", "content": "你是一个强大的聊天机器人"}, {"role": "user", "content": "慕课LLMOps项目中使用什么鉴权方式"}, {"role": "assistant", "content": "使用了Github OAuth作为整个项目的鉴权"}]}
{"messages": [{"role": "system", "content": "你是一个强大的聊天机器人"}, {"role": "user", "content": "LLMOps项目中工作流节点类型有哪些呢？"}, {"role": "assistant", "content": "目前LLMOps项目工作流中支持的节点类型有：开始、Python代码、知识库检索、LLM、HTTP请求、模板转换、扩展插件、结束节点共计8个类型的节点"}]}
{"messages": [{"role": "system", "content": "你是一个强大的聊天机器人"}, {"role": "user", "content": "在慕课LLMOps项目中，怎么快速接入自定义大语言模型呢？"}, {"role": "assistant", "content": "LLMOps使用模型管理器+yaml配置来动态加载不同的LLM，可以在internal/core/language_models下添加该模型提供商，并创建模型类完成映射即可"}]}
{"messages": [{"role": "system", "content": "你是一个强大的聊天机器人"}, {"role": "user", "content": "在LLMOps的Agent应用编排中，支持哪些功能？"}, {"role": "assistant", "content": "Agent编排中支持通过可视化的方式绑定工具、知识库、工作流，并开启语音输入输出、审核配置、开场白建议问题、开场白、使用不同的模型作为Agent的大脑、长短期记忆配置等功能。"}]}
{"messages": [{"role": "system", "content": "你是一个强大的聊天机器人"}, {"role": "user", "content": "在LLMOps项目的docker-compose中，使用了哪些服务"}, {"role": "assistant", "content": "使用了nginx、api、redis、postgres、weaviate向量数据库、ui、celery等服务"}]}
{"messages": [{"role": "system", "content": "你是一个强大的聊天机器人"}, {"role": "user", "content": "在LLMOps项目中，哪个模块使用了Celery异步任务队列"}, {"role": "assistant", "content": "知识库的文档处理功能使用了Celery的异步任务队列功能，并使用redis作为消息代理"}]}
{"messages": [{"role": "system", "content": "你是一个强大的聊天机器人"}, {"role": "user", "content": "在慕课LLMOps项目中为什么使用Flask作为后端框架，而不是Fastapi"}, {"role": "assistant", "content": "Falsk和Fastapi都是优秀的Python后端框架，使用任意的后端框架均可以完成LLMOps项目的开发，没有任何限制"}]}
{"messages": [{"role": "system", "content": "你是一个强大的聊天机器人"}, {"role": "user", "content": "在LLMOps项目中，RAG支持几种优化方式"}, {"role": "assistant", "content": "项目支持向量检索、关键词检索、混合检索，还可以使用工作流编排完成self-rag、crag、父子检索等优化策略"}]}
{"messages": [{"role": "system", "content": "你是一个强大的聊天机器人"}, {"role": "user", "content": "在慕课LLMOps项目中，文档使用什么分割策略"}, {"role": "assistant", "content": "使用LangChain内置的递归字符串文档分割器来分割对应的文档，并且将文档块大小控制在500-1000"}]}
{"messages": [{"role": "system", "content": "你是一个强大的聊天机器人"}, {"role": "user", "content": "在慕课LLMOps项目中，使用了什么embeddings文本嵌入模型"}, {"role": "assistant", "content": "使用了OpenAI提供的Embeddings服务"}]}
```

首先选择右上角的【Create】创建一个新的微调模型，选项配置信息如下：

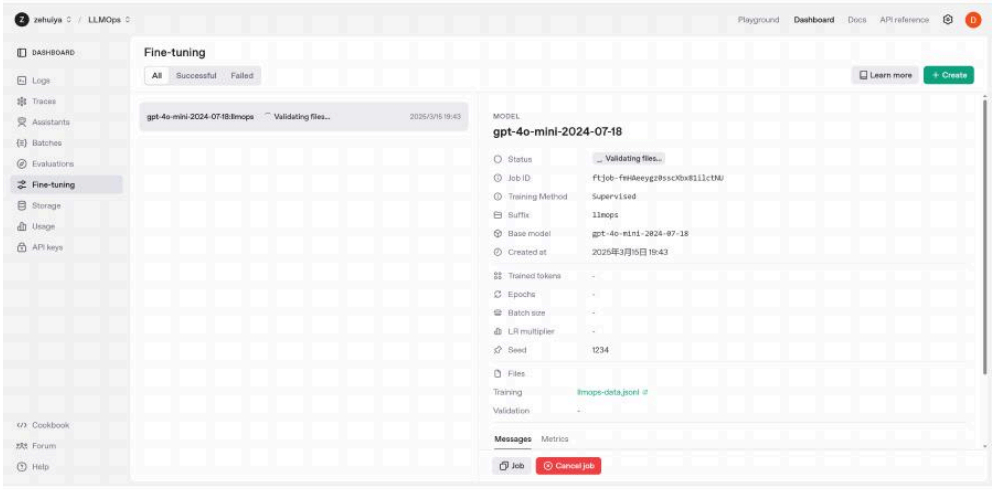
1. 微调方法(Method)：支持Supervised和Direct Preference Optimization两种方法；
2. 基础模型(Base Model)：使用哪个模型进行微调；
3. 训练数据(Training data)：训练数据，支持 `jsonl` 格式；
4. 模型后缀(Suffix)：自定义模型后缀；
5. 随机种子(Seed)：用于控制微调的初始化配置；
6. 批次大小(Batch Size)：每批样本的数量，批大小越大，模型参数更新频率越低；
7. 学习率乘数(Learning rate multiplier)：学习率的缩放因子，较小的学习率可以避免过拟合；

8. 周期数(Number of epochs)：训练模型的周期数，周期是指训练数据集的一个完整周期；

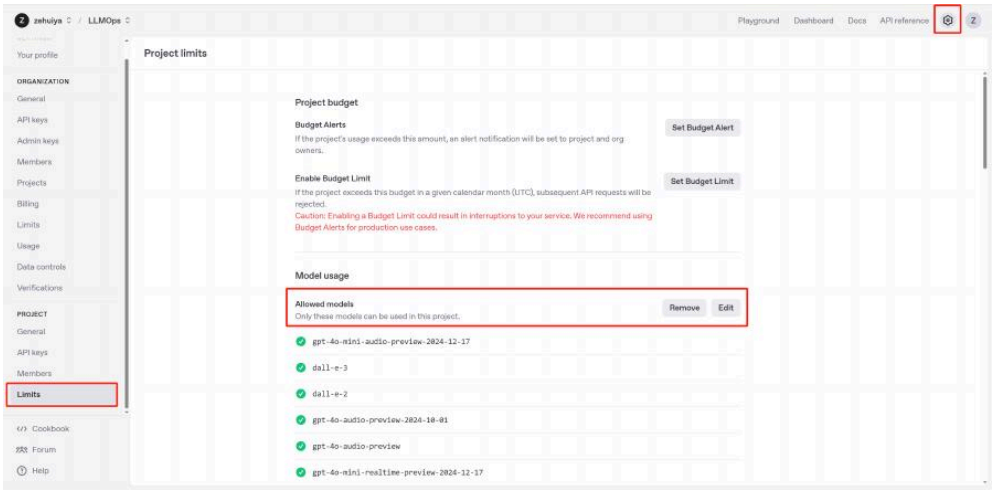
界面配置如下：



选择添加后，就可以看到多个步骤，例如：验证数据、开始微调、微调的不同步骤等信息，如下。



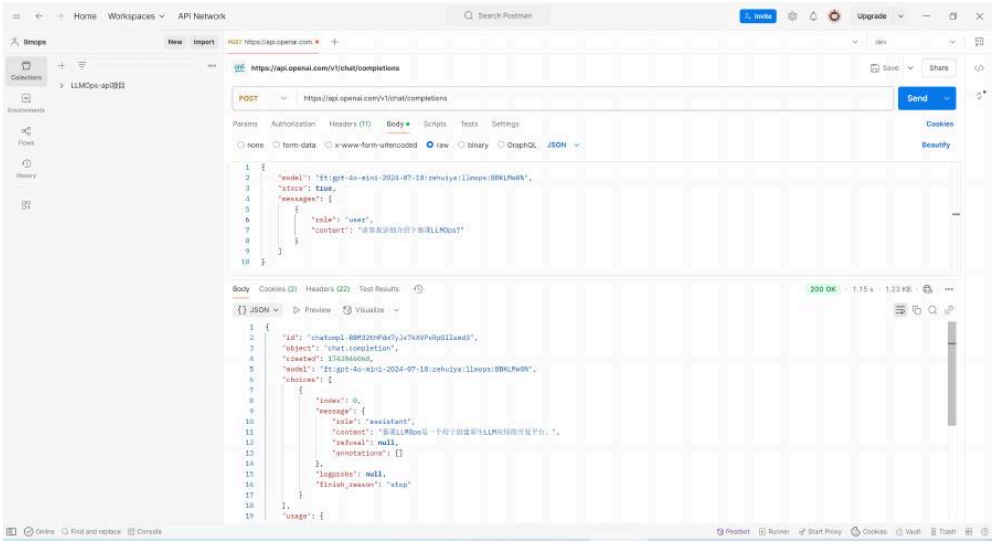
等待训练完成后，就可以看到对应的模型名字、训练消耗的 Token 数（因为数据量较小，总消费为 $0.02），并且默认微调好的模型有可能在当前项目中没法使用，这是因为需要在 `项目设置` 中配置 `允许访问的模型列表`，如下：



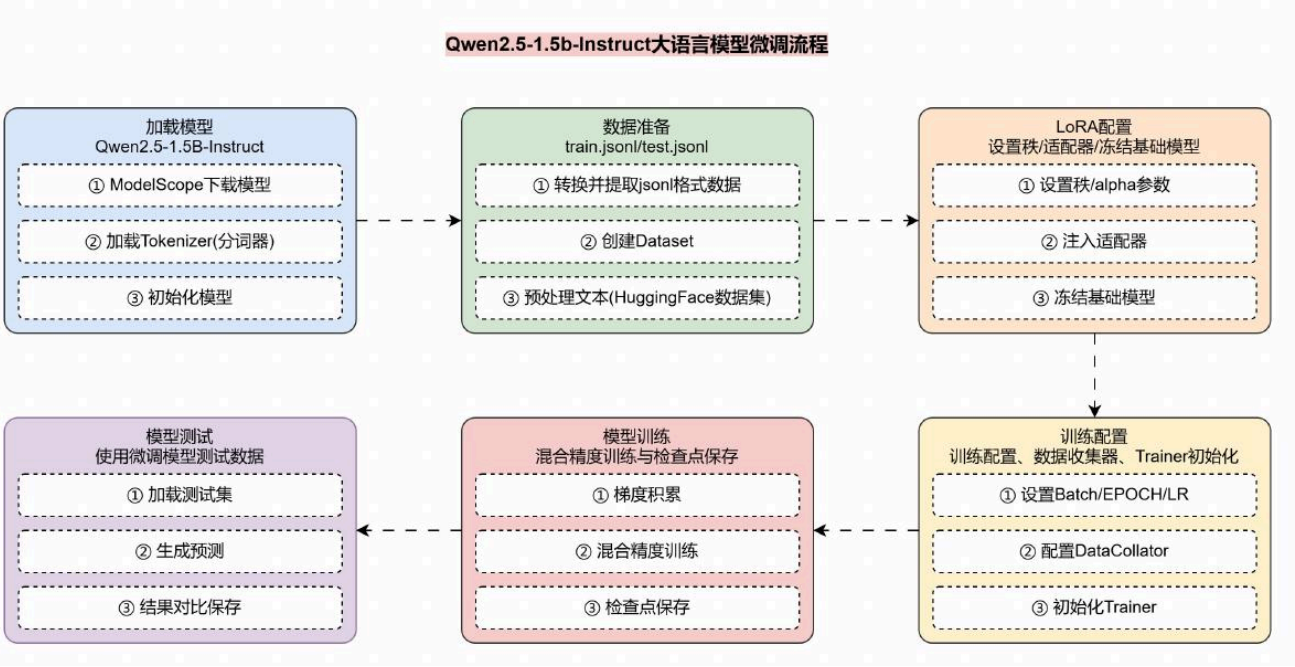
接下来就可以在 **Playground** 中使用微调的模型进行对话，亦或者通过 API 接口使用该微调模型，提问关于 `慕课LLMOps` 的相关问题，使用微调和未经过微调的模型进行回复，可以发现微调模型已经学会了新的知识了，如下：



亦或者是通过 API 进行访问，效果也是一模一样的：



并且如果一次微调效果不好，还可以使用 `微调后的模型` 作为基座模型继续微调，直到达到理想的效果，所以在使用 OpenAI 的云端微调功能时，数据一次使用几千条即可，这样可以及时观察模型是否掌握了这部分知识。


Qwen2.5-1.5b-Instruct大语言模型微调流程
加载模型
Qwen2.5-1.5B-Instruct
① ModelScope下载模型
② 加载Tokenizer(分词器)
③ 初始化模型
数据准备
train.jsonl/test.jsonl
① 转换并提取jsonl格式数据
② 创建Dataset
③ 预处理文本(HuggingFace数据集)
LoRA配置
设置秩/适配器/冻结基础模型
① 设置秩/alpha参数
② 注入适配器
③ 冻结基础模型
训练配置
训练配置、数据收集器、Trainer初始化
① 设置Batch/EPOCH/LR
② 配置DataCollator
③ 初始化Trainer
模型训练
混合精度训练与检查点保存
① 梯度积累
② 混合精度训练
③ 检查点保存
模型测试
使用微调模型测试数据
① 加载测试集
② 生成预测
③ 结果对比保存


扩展学习-LLM的预训练&微调初步了解
01. 预训练&微调&RAG差异
1.1 学习了解预训练、微调与RAG的差异，以及在不同业务下如何选择不同的方案。
1.2 学习了解通过OpenAI官方提供的微调UI快速实现微调GPT模型。
02. 预训练&微调演示
1.1 通过一个示例学习在本地如何通过6个步骤微调LLM，并在课时中演示了微调Qwen2.5-1.5b模型，实现回复指定问题。
1.2 通过一个示例学习在本地如何通过预设配置+训练数据从零开始训练一个大语言模型。